# stringr cheat sheet

### MC 451 / MC 501 / MC 427 · measure and change text

**stringr** is the set of tools for working with text: measuring it, testing it, and changing it. Every stringr function starts with `str_`. Each entry says what it does, shows it on the course Twitch messages, and shows the exact output. **On your data** rewrites the call with a plain column name, and **If it breaks** names the likely error.

Every worksheet starts with these lines, so the examples assume them:

```
library(tidyverse)   # stringr comes with the tidyverse
library(v2v)          # the course package and its Twitch data
```

## The data these examples use

Five chat messages, with their text:

```
chat %>%
  select(channel, message) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 2
  channel    message
  <chr>      <chr>
1 sodapoppin ??????????????????????
2 xqcow      TriEasy Clap TriEasy Clap TriEasy Clap TriEasy Clap TriEasy Cl…
3 forsen     cmonBruh
4 sodapoppin sodaTHINKING
5 xqcow      @xQcOW PLAY JACKBOX PARTY GAMES PLAY JACKBOX PARTY GAMES PLAY …
```

## `str_length()`: count the characters

Returns the number of characters in each string. This is how the wrangling worksheet builds `message_length`.

```
chat %>%
  mutate(message_length = str_length(message)) %>%
  select(message, message_length) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 2
  message                                                    message_length
  <chr>                                                               <int>
1 ??????????????????????                                                 23
2 TriEasy Clap TriEasy Clap TriEasy Clap TriEasy Clap TriEas…            441
3 cmonBruh                                                                 8
4 sodaTHINKING                                                            12
5 @xQcOW PLAY JACKBOX PARTY GAMES PLAY JACKBOX PARTY GAMES P…            206
```

**On your data:** `mutate(length = str_length(text_column))`

**If it breaks:** an empty message gives `NA`, not `0`, when the cell itself is missing. Count the blanks with `count(is.na(message))` before you trust a mean.

## `str_detect()`: does the text contain a pattern?

Returns `TRUE` or `FALSE` for each string, saying whether it holds the text you looked for. Pair it with `count()` to tally the matches.

```
chat %>%
  mutate(has_clap = str_detect(message, "Clap")) %>%
  count(has_clap)
```

```
# A tibble: 3 × 2
  has_clap     n
  <lgl>    <int>
1 FALSE    35003
2 TRUE       260
3 NA           4
```

**On your data:** `mutate(flag = str_detect(text_column, "word"))`

**If it breaks:** the match is case-sensitive, so `"clap"` and `"Clap"` differ. Use `str_detect(message, regex("clap", ignore_case = TRUE))` to catch both.

## `str_replace()`: swap the first match

Replaces the first place a pattern appears in each string with new text. Use `str_replace_all()` to replace every match, not just the first.

```
str_replace(c("sodaTHINKING", "cmonBruh"), "THINKING", "_thinking")
```

```
[1] "soda_thinking" "cmonBruh"
```

**On your data:** `str_replace(text_column, "old", "new")`

**If it breaks:** nothing changes when the pattern is not found; the original string comes back unchanged, as `cmonBruh` does above.

## `str_c()`: join pieces of text end to end

Pastes strings together. Any plain text you give is repeated for every row.

```
str_c("channel: ", c("bobross", "xqcow"))
```

```
[1] "channel: bobross" "channel: xqcow"
```

**On your data:** `str_c(first_name, " ", last_name)`

**If it breaks:** a gap you did not want means you need a space inside the quotes, as in `": "` above. `str_c()` adds nothing between pieces on its own.

## `str_glue()`: build a sentence from values

Builds a string by dropping values into `{ }` inside the text, which reads more clearly than chaining `str_c()`.

```
name <- "bobross"
n <- 847
str_glue("{name} had {n} messages")
```

```
bobross had 847 messages
```

**On your data:** `str_glue("{label}: {value}")`

**If it breaks:** `object 'name' not found` means a name in the braces does not exist yet. Every `{ }` must hold something R already knows.

## `word()`: pull out a word by position

Returns the word at a position, counting from 1. Useful for a first name, a first tag, or the lead word of a title.

```
word(c("Hello truckers", "RANK 1 GAMER"), 1)
```

```
[1] "Hello" "RANK"
```

**On your data:** `word(text_column, 1)` for the first word, `word(text_column, -1)` for the last.

**If it breaks:** asking for word 3 of a two-word string returns `NA`. Check the position exists.

## `str_to_lower()`: make text lowercase

Lowercases every letter, which is how you stop `Bob` and `bob` from counting as two different values before a `count()` or a join.

```
str_to_lower(c("TriEasy", "CmonBruh"))
```

```
[1] "trieasy"  "cmonbruh"
```

**On your data:** `mutate(key = str_to_lower(text_column))`

**If it breaks:** nothing to break. Use `str_to_upper()` for capitals and `str_to_title()` for Title Case.